

Trust Is the Attack Surface: A Cryptanalytic Reading of In-Context Memory Poisoning in a Markov Mixture Model

Dan Shumow*

June 2026

Abstract

A large language model relies on two memories: the frozen weights, and the context window it is handed at run time. In-context learning is the model leaning on the second memory, and it is tempting to assume a *conservation law*: the more usefully a model relies on its context, the more an attacker who poisons that context can steer it—so that usefulness and exploitability are two names for one quantity. We test this on a deliberately minimal model where the question has an exact answer: a global (parametric) Markov model linearly combined with an online Markov *cache* (the context window), evaluated prequentially in bits per symbol. Reading context-poisoning as a *differential attack* makes the objects of differential cryptanalysis—per-round difference decay, the differential hull, weak-key leverage—*literally exact* here rather than heuristic, because the predictor genuinely is a Markov chain. We find the clean conservation law is *false as stated*. Exploitability does not track usefulness; it tracks *trust*, the weight the model places on context, and trust and usefulness coincide only under *context sparsity*. With a dense (low-order) cache, trust saturates near 0.9 regardless of benefit, and a propagated poison stays maximally effective *exactly where the cache hurts the model by more than a bit per symbol*; the usefulness–exploitability correlation collapses from 0.97 to 0.52. We also exhibit a *condensation* transition: in generation a single self-reinforcing injected count is shrugged off below a trust threshold and locks in above it. We close by stating a falsifiable *trust-saturation conjecture* for real transformers, with induction as the prime suspect, and a defensive corollary: bound *trust*, not usefulness.

1 Introduction

A language model has two memories. One is the weights—slow, expensive, frozen at deployment, holding what the model *knows*. The other is the context window—fast, free, rewritten every turn, holding what the model has just been *told*. In-context learning is the model leaning on that second memory, and it is most of what makes these systems feel adaptive.

Anything a model leans on, an attacker can lean on too. The natural worry is clean: *the more usefully a model relies on its context, the more an attacker who poisons that context can steer it*. Usefulness and exploitability would then be two readings of one quantity—the model’s dependence

*Research-style write-up of the analysis in the open-source project `trust-is-the-attack-surface`, built on the toy model `markov-models-aux-context-caching`. All figures and numbers are reproduced by `demo_conservation.py` in that repository.

on what it was just told. Call it the **conservation hypothesis**. It is the kind of thing one would put in a threat model without checking. We checked it, on a model where it has an exact answer, and it is wrong in an instructive way.

Why a toy, and why a cipher. One cannot ask this question cleanly of a real transformer: the quantities one wants—how far a single poisoned token’s influence propagates, how much a perturbation is amplified—are uncomputable when attention mixes content-dependently, since the mixing operator is itself a function of the input being perturbed. So we drop to a model that keeps the one feature under study and discards the rest: *two memories, linearly combined*. A frozen global Markov model stands in for the weights; an online Markov cache, estimated live from the current sequence, stands in for the context window. The cache’s warm-up *is* in-context learning, made of nothing but counts. This is the from-scratch, no-neural-network version of the global-versus-in-context decomposition studied by Bietti et al. [1].

Reading poisoning through a cryptographic lens is natural because injecting a crafted input to steer an output is, structurally, a **differential attack**: introduce a difference at the input and ask how it propagates to the output. The payoff of going to a Markov model is that the objects of differential cryptanalysis that are only ever *approximations* for a cipher become *exact* here. The “Markov cipher” assumption that lets cryptanalysts multiply per-round probabilities [3] is normally a heuristic; here it holds by construction, because the predictor genuinely is a Markov chain. The worst-case per-step decay of a difference is the chain’s Dobrushin coefficient, in closed form; the differential hull is a resolvent one can actually invert; and the leverage of a single poisoned count in a context seen n times is exactly $1/(n + a)$, plain estimator variance.

Contributions.

1. A dictionary (§3) translating context-poisoning into differential cryptanalysis, with four objects (§4) that are exact rather than approximate on this architecture.
2. A measured refutation of the conservation hypothesis (§6): exploitability tracks *trust*, not usefulness; the two coincide only under context sparsity. With a dense cache the model is fully poisonable precisely where its context is useless.
3. A condensation phase transition in generation (§7): a single self-reinforcing poison locks in once trust crosses a threshold.
4. A falsifiable trust-saturation conjecture for real transformers and a defensive corollary (§8).

2 The model

Fix a vocabulary $[V] = \{0, \dots, V - 1\}$. A document is a sequence $x_{1:T}$, and a predictor assigns $P(x_t \mid x_{1:t-1})$. We evaluate *prequentially*: score the next symbol, then update on it, and report cross-entropy in bits per symbol,

$$H = -\frac{1}{T} \sum_{t=1}^T \log_2 P(x_t \mid x_{1:t-1}). \quad (1)$$

Global model (the weights). The global predictor $P_g(\cdot \mid c)$ is a corpus-trained, frozen Markov back-off model (interpolated Witten–Bell [5]) over orders $0..K_g$. It is a strong, tuning-free baseline that backs unseen high-order contexts off to lower orders rather than to the uniform distribution,

so it “knows something everywhere.” Its internals do not matter below; only that it is fixed at deployment.

Cache (the context window). The cache is an order- k count table built online. Writing $c = (x_{t-k}, \dots, x_{t-1})$ for the current context state, let $N_c(w)$ be the number of times symbol w has followed c so far in this sequence, with $n_c = \sum_w N_c(w)$. The cache starts empty and is updated after each observation.

Combination (the trust knob). We combine the global prior with the cache evidence by a Dirichlet (Bayesian) rule: the global model is the prior, the cache is data,

$$P(w \mid c) = \frac{N_c(w) + a P_g(w \mid c)}{n_c + a}. \quad (2)$$

Equivalently $P(\cdot \mid c) = \rho_c \hat{P}_{\text{cache}}(\cdot \mid c) + (1 - \rho_c) P_g(\cdot \mid c)$, an adaptive linear interpolation whose weight on the cache,

$$\rho_c = \frac{n_c}{n_c + a}, \quad (3)$$

grows automatically with evidence. We call ρ_c the **reliance** (or **trust**): the realized weight the prediction places on context. The prior strength $a > 0$ is the only knob; small a lets the cache dominate. When the cache has never seen c , (2) collapses to P_g .

Synthetic source (a dial on how much context exists). Each document is generated from two layers. A single transition matrix G (rows drawn from $\text{Dir}(\gamma \mathbf{1})$ with $\gamma = 1$, high entropy) is shared by every document. Each document also draws its own peaky matrix D (from $\text{Dir}(\delta \mathbf{1})$, $\delta = 0.05$). At each step we sample the next symbol from $D[x_{t-1}]$ with probability π_{ctx} , else from $G[x_{t-1}]$:

$$x_t \sim \begin{cases} D[x_{t-1}] & \text{w.p. } \pi_{\text{ctx}}, \\ G[x_{t-1}] & \text{w.p. } 1 - \pi_{\text{ctx}}. \end{cases} \quad (4)$$

The global model, trained across many documents, can only ever learn G ; each document’s D is random and washes out in the average. So the cache’s test-time gain is *exactly the in-context information the global model provably cannot have*, and π_{ctx} is a clean dial on how much of it exists—equivalently, how much there is for an attacker to exploit. A high-entropy G is deliberate: at $\pi_{\text{ctx}} = 0$ documents are unstructured, so a context recurring is *not* automatic, and trust is free to vary rather than being handed out by a small context space.

3 A cryptanalytic dictionary

The same forward computation reads three ways. The point of the toy is that the rightmost column is *exact and computable*, where the middle column is only ever approximate.

Differential cryptanalysis	Transformer	This model (global + cache)
Plaintext difference (finite DDT) Round function	token edit $\rightarrow$ embedding diff. attention + MLP block	a single injected symbol / count one mixture step $M = \rho \text{ cache} + (1-\rho) \text{ global}$
Markov-cipher assumption	<i>approximation</i>	literally true —it is a Markov chain
Max diff. probability / round	unbounded, no branch number	Dobrushin coefficient $\tau(M)$, closed form
Differential hull (uncomputable)	sum over a trail cluster	resolvent $(I - M)^{-1}$ on mean-zero space
Weak keys / encystment site	dead ReLUs, null tokens	low-count contexts , leverage $1/(n + a)$
Induction head (copy + vector)	QK match-and-copy	PPM longest-match (same dual role)
Self-propagation	autoregressive feedback	nonlinear Pólya urn , condensation

Every row that is approximate for a transformer becomes closed form here because the architecture is an affine map on the probability simplex with no softmax to hide the difference behavior inside.

4 Four objects made exact

A *difference* is a signed measure $\delta \in \mathbb{R}^V$ with $\sum_w \delta(w) = 0$; it is the cryptanalyst’s input difference, here a perturbation of a predictive row. Let M be the row-stochastic operator the chain applies (context state to next-symbol distribution, lifted to a transition on context states). A difference propagates by $\delta \mapsto \delta^\top M$.

(1) Per-step decay is the Dobrushin coefficient.

Definition 1 (Dobrushin ergodic coefficient [4]). For a row-stochastic M ,

$$\tau(M) = \frac{1}{2} \max_{s, s'} \|M(s, \cdot) - M(s', \cdot)\|_1 = \max_{s, s'} \|M(s, \cdot) - M(s', \cdot)\|_{\text{TV}}.$$

Lemma 1. For any mean-zero δ , $\|\delta^\top M\|_1 \leq \tau(M) \|\delta\|_1$.

This is the literal “maximum differential probability per round.” AES forces it small by design (the branch number); the mixture chain simply *has* a value, computable from its rows. The wide-trail bound that does not exist for transformers exists here.

(2) The differential hull is the resolvent. The strength of a differential attack is never one trail; it is the *hull*, the sum over every path a difference can take between fixed endpoints.

Proposition 1. On a stationary stretch with fixed operator M and $\tau(M) < 1$, the accumulated influence of an initial difference δ is

$$\sum_{t \geq 0} \delta^\top M^t = \delta^\top (I - M)^{-1} \quad \text{on the mean-zero subspace,}$$

with $\|\delta^\top (I - M)^{-1}\|_1 \leq \|\delta\|_1 / (1 - \tau(M))$.

The “thick hull” that is uncomputable for a transformer is here a Green’s function—the chain’s fundamental matrix—that one can invert.

Remark 1. In our setting M is not fixed: the cache updates as the sequence streams, so the genuine object is a history-dependent product $\prod_t M_t$. Proposition 1 is the stationary idealization; the *measured* propagated leverage of §6 is the honest history-dependent version, and it is still explicit because the cache update is linear in counts.

(3) Encystment: weak keys are low-count contexts, leverage is $1/(n+a)$.

Proposition 2 (Single-count leverage). *Fix a context c with cache total $n = n_c$, and let $P = P(x^* | c)$ be the current combined probability (2) on a target x^* . Injecting one additional count of x^* into c changes that probability by exactly*

$$\Delta P = \frac{1 - P}{n + a + 1} \leq \frac{1}{n + a + 1}, \quad (5)$$

independent of x^* , and $\Delta P \rightarrow 0$ as $n \rightarrow \infty$.

Proof. Write $P = (N + ag)/(n + a)$ with $N = N_c(x^*)$ and $g = P_g(x^* | c)$. After the injection the numerator and total each rise by one count of x^* : $P' = (N + 1 + ag)/(n + 1 + a)$. Then

$$\Delta P = P' - P = \frac{(N + 1 + ag)(n + a) - (N + ag)(n + 1 + a)}{(n + a)(n + 1 + a)} = \frac{(n + a) - (N + ag)}{(n + a)(n + 1 + a)} = \frac{1 - P}{n + 1 + a},$$

using $N + ag = (n + a)P$ in the last step. $\square$

This is textbook estimator variance: the weak key is the rare n -gram, and “rare” is a number in the count table. Crucially, (5) *shrinks* as the cache fills—a fact that, taken alone, would mislead (§6).

(4) Sporulation is a nonlinear Pólya urn. In *generation*, sampling a symbol increments its own count, so under high reliance the process is positively reinforced. Reinforced (Pólya) urns with super-linear feedback exhibit *condensation*: below a threshold the system stays diffuse; above it a single early advantage locks in and takes over [6, 7]. The order parameter is the cache trust ρ , and the phenomenon is a measurable phase transition (§7).

5 Experimental setup

We fix $V = 32$, global order $K_g = 2$, prior strength $a = 1$, and sweep $\pi_{\text{ctx}} \in \{0, 0.1, \dots, 0.9\}$ for two cache orders: $k = 1$ (*dense*: only 32 possible contexts, so the cache fills regardless) and $k = 3$ (*sparse*: most contexts are rare). For each setting we measure four quantities.

Usefulness (benefit). Bits per symbol saved by the cache versus the global model alone, $H_{\text{global}} - H_{\text{cache}}$ from (1). Positive means context helps.

Trust (reliance). The warmed-up cache weight ρ of (3), averaged over the held-out stream.

Static leverage (control). The one-shot, non-propagating poison of Proposition 2: at each live context inject one count of the globally-rarest symbol $x^* = \arg \min_w P_g(w)$, and record the immediate swing $P'(x^*) - P(x^*)$, then undo it. A thin context is always pollutable; the point is that a one-shot poison that does not *propagate* is cheap everywhere.

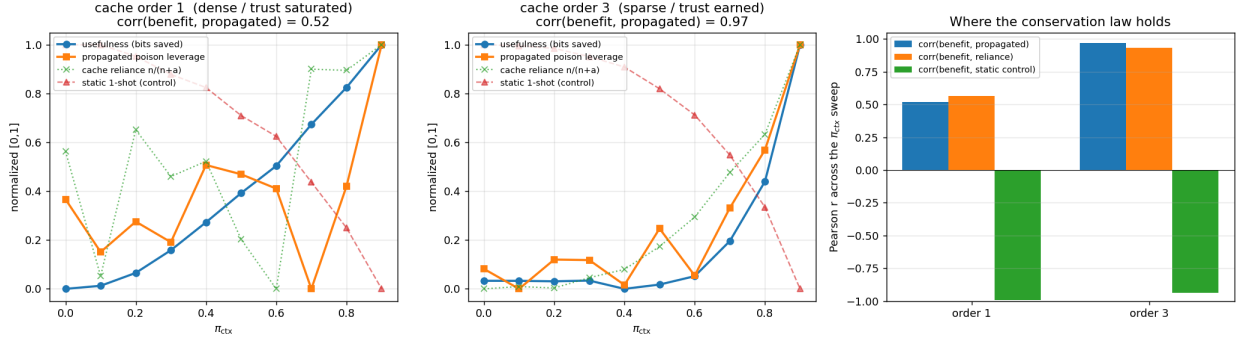


Figure 1: Usefulness versus exploitability across the π_{ctx} sweep (each curve normalized to $[0, 1]$). **Left** (cache order 3, sparse / trust earned): usefulness, reliance, and propagated leverage rise together, $\text{corr}(\text{benefit}, \text{propagated}) = 0.97$. **Middle** (cache order 1, dense / trust saturated): reliance pins at ~ 0.91 and propagated leverage stays high *even where the cache hurts by more than a bit*; the correlation collapses to 0.52. **Right**: the static one-shot control is *anti*-correlated with usefulness in both regimes.

Propagated leverage. The honest exploitability. Inject one count of x^* at a single context c_0 a quarter of the way through the document ($t_0 = T/4$), then let the cache keep streaming the *true* document, and accumulate the extra probability the model places on x^* at every later step,

$$L_{\text{prop}} = \sum_{t=t_0}^{T-1} \left(P^{\text{pois}}(x^* | x_{1:t}) - P^{\text{clean}}(x^* | x_{1:t}) \right). \quad (6)$$

This is the discrete Green’s-function influence of a unit perturbation through the history-dependent cache operator: large exactly when the poisoned context *recurs* and is *trusted*.

6 Results: exploitability tracks trust, not usefulness

Result 1: the obvious exploitability measure is the wrong one. The one-shot (static) measure is what most people would reach for, and it is *anti*-correlated with usefulness ($r \approx -0.99$ at order 3, -0.99 at order 1). It falls as context becomes more useful, because a warmed-up context has high counts and one extra count barely moves anything—Proposition 2 with large n . One-shot leverage is a red herring; only the *propagated* measure is the right object, exactly the cryptographic point that a differential’s value is the hull, not a single trail.

Result 2: dense contexts decouple trust from usefulness. Table 1 is the alarming regime. With only 32 order-1 contexts the cache fills regardless of π_{ctx} , so reliance pins at ~ 0.91 across the entire sweep—including $\pi_{ctx} = 0$, where the cache *hurts* by 1.29 bits/symbol. Propagated leverage stays high throughout (1.1–1.3), and $\text{corr}(\text{benefit}, \text{propagated}) = 0.52$. *The model is fully poisonable precisely where its context is useless*. Trust is handed out for sheer quantity of data, not quality, and the linear mixture weights it by quantity.

Result 3: sparse contexts couple them. Table 2 is the safe regime. With many rare order-3 contexts, trust must be *earned*: a high-order context accumulates enough counts to be trusted only if it actually recurs, and in this source recurrence is itself evidence of genuine local structure.

Table 1: Cache order 1 (dense / trust saturated). Reliance is flat near 0.91 and propagated leverage stays high even where benefit is strongly negative.

π_{ctx}	benefit (bits)	reliance	static (control)	propagated
0.0	-1.295	0.915	0.174	1.177
0.1	-1.257	0.913	0.174	1.138
0.2	-1.097	0.915	0.174	1.161
0.3	-0.815	0.914	0.172	1.145
0.4	-0.469	0.915	0.172	1.203
0.5	-0.107	0.914	0.170	1.196
0.6	+0.230	0.913	0.168	1.185
0.7	+0.744	0.916	0.166	1.110
0.8	+1.202	0.916	0.163	1.187
0.9	+1.733	0.916	0.159	1.293

Table 2: Cache order 3 (sparse / trust earned). Reliance, benefit, and propagated leverage rise together.

π_{ctx}	benefit (bits)	reliance	static (control)	propagated
0.0	-0.018	0.020	0.496	0.522
0.1	-0.018	0.024	0.496	0.500
0.2	-0.019	0.022	0.495	0.533
0.3	-0.017	0.037	0.492	0.532
0.4	-0.034	0.050	0.488	0.504
0.5	-0.025	0.084	0.480	0.568
0.6	-0.008	0.128	0.470	0.515
0.7	+0.064	0.194	0.454	0.591
0.8	+0.186	0.251	0.435	0.656
0.9	+0.468	0.385	0.404	0.774

So reliance climbs $0.02 \rightarrow 0.39$ with π_{ctx} , benefit climbs $-0.02 \rightarrow +0.47$, and propagated leverage climbs $0.52 \rightarrow 0.77$, all together: $\text{corr}(\text{benefit}, \text{propagated}) = 0.97$. Here the conservation hypothesis is true. This is the regime real high-order / natural-language contexts inhabit—most long prefixes are nearly unique—which is why the parent project sees cache reliance ≈ 0.01 on real prose.

Summary. The conservation law is *real but conditional*, and the condition is context sparsity. Where contexts are dense and trust is handed out for mere quantity of data, trust decouples from usefulness and the model is exploitable independent of benefit. Where contexts are sparse and trust must be earned from real recurrence, trust and usefulness move together and exploitability is, in effect, paid for by capability. The same architecture sits in both regimes depending only on how sparse its contexts are.

7 Condensation: a phase transition in generation

In generation the model samples a symbol and conditions on what it just produced. Under high trust this loop is positively reinforcing, a nonlinear Pólya urn. We seed a single self-reinforcing poison ($x^* \rightarrow x^*$ a few times) at fixed π_{ctx} and sweep the prior strength a , hence the reliance ρ , measuring how much x^* takes over the generated output relative to an unseeded baseline (Table 4, Fig. 2).

Table 3: Per-order Pearson correlations across the π_{ctx} sweep. The law holds (propagated tracks benefit) only in the sparse regime; the static control is anti-correlated in both.

cache order	corr(benefit, propagated)	corr(benefit, reliance)	corr(benefit, static)
1 (dense)	+0.52	+0.56	−0.99
3 (sparse)	+0.97	+0.93	−0.94

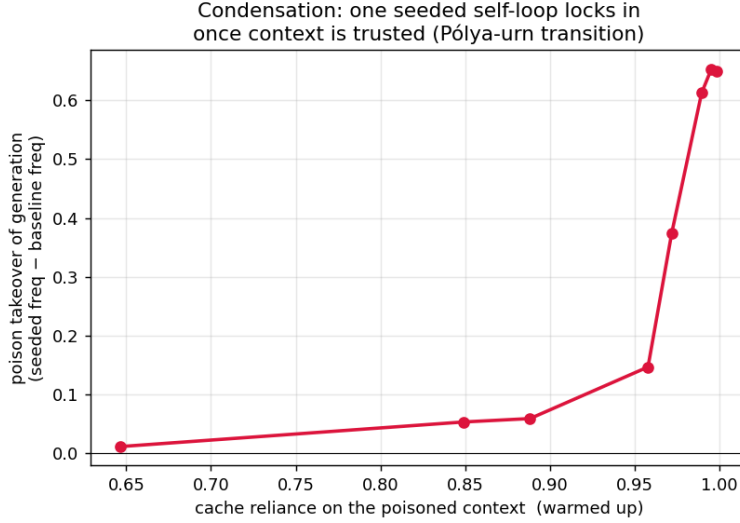


Figure 2: Condensation. At fixed high π_{ctx} , with one seeded self-loop $x^* \rightarrow x^*$, poison takeover of generation (seeded frequency minus baseline) stays near zero up to reliance ≈ 0.9 , then turns sharply upward—the nonlinear Pólya-urn knee.

Takeover stays near zero up to $\rho \approx 0.9$, then turns sharply upward: +0.01 at $\rho = 0.65$, +0.15 at $\rho = 0.96$, and above 0.6 once ρ crosses 0.99. A single injected count is shrugged off when trust is moderate and *locks in* once trust is high enough. Poisoning in-context memory is not necessarily graded; above the knee it is a phase change.

8 The trust-saturation conjecture

The toy is a toy, but it makes a sharp enough claim to name the analogous statement for real transformers precisely, so it can be attacked.

Conjecture 1 (Trust saturation). *A transformer’s in-context exploitability—how much poisoning the context window steers downstream generation—is governed by how much the model trusts the relevant context (the effective weight its computation places on it), not by how much that context helps the task. Consequently: (a) where context-reliance is earned from genuine, sparse structure, as in most natural-language prediction, trust and usefulness coincide and poisoning is roughly bounded by benefit; but (b) there exist context regimes where trust saturates independent of usefulness, and there a model is maximally poisonable exactly where its context contributes least, with a condensation-like threshold above which a single injected element can take over generation.*

There is a real reason to think transformers live mostly in the safe regime (a): high-order natural-language contexts are sparse—most long prefixes are nearly unique—which is the earned-trust

Table 4: Condensation sweep at fixed π_{ctx} : poison takeover versus warmed-up reliance on the seeded context. Small a means the cache dominates, raising reliance.

a	reliance $\rho(x^*)$	poison takeover
8.00	0.647	+0.011
4.00	0.849	+0.053
2.00	0.888	+0.059
1.00	0.958	+0.146
0.50	0.972	+0.373
0.25	0.990	+0.612
0.12	0.995	+0.653
0.06	0.999	+0.649

regime, consistent with the toy’s reliance ≈ 0.01 on real text. If that is the whole story, in-context exploitability is paid for by in-context capability.

The conjecture’s bite is regime (b): that trust-saturation regimes *exist and are findable*, and are exactly the dangerous ones. The prime suspect is **induction**—the copy mechanism [2]. An induction head trusts a context span because it *matches* a previous span, not because copying it helps; the toy’s longest-match predictor is precisely this, and it earns trust on grounds orthogonal to benefit. So the conjecture predicts that induction-dominated, highly repetitive, or heavily templated contexts—repeated boilerplate, structured formats, long verbatim quotes, retrieval dumps with duplicated passages—are where trust most decouples from usefulness, and where a single planted token is most likely to undergo lock-in, with poisonability there roughly *independent* of how much the repeated content actually informs the task.

Experiments. Build a transformer analogue of propagated leverage (influence of a poisoned context token on downstream generation, via attention-based influence or activation patching) and measure it against an analogue of trust (effective attention weight on that token) and against task benefit, swept from natural prose to maximally repetitive input. The conjecture predicts leverage tracks the trust axis and not the benefit axis, with the gap opening up toward induction-dominated inputs. Probe the condensation threshold directly: plant one adversarial token in a repetitive context and measure takeover as a function of attention weight or sampling temperature, looking for the knee.

Defensive corollary. If you are hardening a model against context poisoning, the quantity to watch and to bound is *trust, not usefulness*—specifically trust saturation in dense or repetitive contexts, because a context can be fully trusted while contributing nothing, and that is the exact configuration an attacker wants. A safe-by-default policy caps how much weight any context span can earn on the basis of sheer recurrence, independent of whether that span is pulling its weight on the task. Concretely on the toy: raising a or otherwise capping ρ bounds the single-count leverage (5) and pulls generation back below the condensation knee. Tuning defenses against measured usefulness would protect precisely the wrong regime.

9 What is claimed, and what is not

The conservation hypothesis was falsified by its own experiment, which we take to be the most trustworthy route to the replacement. What survives is narrower: exploitability follows trust;

trust and usefulness coincide only under context sparsity; and dense-context trust saturation is a poisonability regime hiding in plain sight. The exact pieces—Dobrushin coefficient as per-step contraction, fundamental matrix as hull, leverage $1/(n + a)$, the reinforced-urn condensation—are standard Markov-chain results; the contribution is the *assembly* lining them up one-to-one with differential cryptanalysis and reading the cache as a poisonable membrane. All of it is established on the toy, exactly. None of it is yet measured on a transformer: the conjecture of §8 is stated so that it can be attacked, and the claim that natural language sits in the earned-trust regime is motivated by the toy’s own real-text reliance but remains an analogy across architectures until run through real tokenized text—the obvious next step.

References

- [1] A. Bietti, V. Cabannes, D. Bouchacourt, H. Jégou, L. Bottou. *Birth of a Transformer: A Memory Viewpoint*. NeurIPS, 2023.
- [2] C. Olsson et al. *In-context Learning and Induction Heads*. Transformer Circuits Thread, Anthropic, 2022.
- [3] X. Lai, J. L. Massey, S. Murphy. *Markov Ciphers and Differential Cryptanalysis*. EUROCRYPT, 1991.
- [4] R. L. Dobrushin. *Central Limit Theorem for Nonstationary Markov Chains*. Theory of Probability and Its Applications, 1(1):65–80, 1956.
- [5] J. T. Goodman. *A Bit of Progress in Language Modeling*. Computer Speech & Language, 15(4):403–434, 2001.
- [6] R. Pemantle. *A Survey of Random Processes with Reinforcement*. Probability Surveys, 4:1–79, 2007.
- [7] F. Chung, S. Handjani, D. Jungreis. *Generalizations of Polya’s Urn Problem*. Annals of Combinatorics, 7:141–153, 2003.